

Bootstrapping GSW and TFHE

Lecture 15

SEC 595 — Encrypted Computing

- 1 The Idea
- 2 GSW Decryption as a Circuit
- 3 The Evaluation Key
- 4 Noise
- 5 TFHE

Bootstrapping (Lecture 14) changes one argument of Eval:

$$\Pi \longmapsto C_{\text{Dec}},$$

the scheme's own decryption circuit.

The key enters encrypted, so the output is not the message but a *fresh encryption* of it.

Two things must be true:

- 1 C_{Dec} is a circuit GSW can evaluate, with budget to spare;
- 2 the evaluator holds encryptions of the secret key.

A GSW ciphertext (Lecture 13) satisfies

$$C \cdot s = M \cdot Gs + e, \quad \|e\|_\infty < q/4.$$

Only the last entry is used:

$$(C \cdot s)_\ell = \sum_{j=1}^{n+1} C_{\ell j} s_j = M \cdot \frac{q}{2} + e_\ell.$$

M is the top bit: is the value nearer 0 or $q/2$? This is Regev decryption (Lecture 7).

So C_{Dec} is **one inner product of length $n + 1$ and a comparison.**

The inner product over bits

Choose the secret **binary**: $s_j \in \{0, 1\}$ (Lecture 6).

- $C_{\ell j} s_j$: the $\log q$ known bits of $C_{\ell j}$, each ANDed with s_j . One layer of AND gates.
- $\sum_j C_{\ell j} s_j \bmod q$: an adder tree of full adders, each a small AND/XOR circuit.
- The test reads one bit of the result.

The entries $C_{\ell j}$ are known to the evaluator: they are **constants** of the circuit, not encrypted inputs.

Ciphertext operation	On messages	Gate
$G^{-1}(C_1) \cdot C_2$	$M_1 M_2$	AND
$C_1 + C_2 - 2G^{-1}(C_1)C_2$	$M_1 \oplus M_2$	XOR
$G - G^{-1}(C_1)C_2$	$1 - M_1 M_2$	NAND

Plain $C_1 + C_2$ gives $M_1 + M_2$, which is XOR only when $M_1 M_2 = 0$.

NAND alone is universal, so every Boolean circuit — in particular C_{Dec} — is in reach, subject to the noise budget.

Cost of C_{Dec}

Size: on the order of $(n + 1) \log q$ full adders.

AND-depth: $d = O(\log n + \log \log q)$.

Raise q to buy more budget: the budget grows like q , but d grows only like $\log \log q$.

The gap between what decryption costs and what the scheme can afford *widens*. That is why GSW is bootstrappable.

Example — the decryption circuit

$n = 3$, $q = 8$, binary $s \in \{0, 1\}^4$, constants $C_{\ell 1}, \dots, C_{\ell 4}$.

- a. Draw an AND/XOR circuit with inputs $s_1, \dots, s_4$ and one output wire, the message bit.
- b. Count the AND and XOR gates. What is the AND-depth?
- c. Mark the selection layer and the adder tree. Which one adds depth?

The evaluation key is a list of ciphertexts

Bootstrapping key

$$ek = (\text{Enc}(s, s_1), \dots, \text{Enc}(s, s_{n+1}))$$

one GSW ciphertext per (binary) key bit, under that same key.

Refreshing C :

$$\text{Eval}(ek, C_{\text{Dec}_C}, ek) = \tilde{C} \neq C, \quad \text{Dec}(s, \tilde{C}) = \text{Dec}(s, C).$$

C is hard-wired as constants; only the key bits are encrypted inputs.
Constants cost no gates and no noise.

Publishing $\text{Enc}(s, s_i)$ is not covered by IND-CPA (Lecture 8): there the messages never depend on the key.

Definition

A scheme is **circular secure** if it stays secure when the adversary also holds encryptions of the secret key under that key.

No reduction from LWE is known, and no attack. It is an extra assumption.

The alternative — a chain of keys, each encrypted under the next — avoids cycles but fixes the number of bootstraps: the scheme becomes *levelled*.

Example — what the key costs

$n = 500$, $q = 2^{60}$, so $\ell = (n + 1) \log q$.

- a. Compute ℓ and the size of one GSW ciphertext ($\ell \times (n + 1)$ entries of 60 bits).
- b. How large is ek with a binary secret?
- c. How large if s is uniform and each entry is encrypted bit by bit?

Recall the asymmetry of GSW multiplication (Lecture 13):

$$G^{-1}(C_1) C_2 : \quad \|e_{\text{new}}\|_{\infty} \leq \|e_1\|_{\infty} + \ell \|e_2\|_{\infty}.$$

Only the *second* operand's noise is multiplied by ℓ .

So evaluate C_{Dec} **sequentially**:

- the running value (the *accumulator*) always in the decomposed slot C_1 ;
- a fresh ciphertext from ek always in C_2 .

Each gate then adds at most ℓE ; nothing is multiplied.

Tracing the noise through one decryption

Fresh ciphertexts in ek have $\|e\|_\infty \leq E$. After i gates:

$$N_0 = E, \quad N_i \leq N_{i-1} + \ell E \implies N_g \leq E(1 + g\ell).$$

The noise of the input C never appears: it is consumed by the decryption test, whose answer is a bit.

Bootstrapping condition

$$\|e_{\tilde{C}}\|_\infty \leq E(1 + g\ell) < \frac{q}{4}.$$

A balanced tree instead gives $E(\ell + 1)^d$ — exponential in the depth.

Example — the bootstrapping budget

$$n = 500, E = 2^{10}.$$

- a. $q = 2^{60}$: find the largest g with $E(1 + g\ell) < q/4$.
- b. Compare with the roughly 30 000 gates of C_{Dec} . Is GSW bootstrappable here?
- c. Repeat (a) for $q = 2^{30}$.
- d. $q = 2^{60}$, balanced tree: largest AND-depth d with $E(\ell + 1)^d < q/4$?

TFHE: bootstrap after every gate

BFV, BGV, CKKS and GSW bootstrap *rarely* and spend a noise budget in between. TFHE does the opposite.

LWE over the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$

Binary key $s \in \{0, 1\}^n$. Encryption of μ : (a, b) with $b = \langle a, s \rangle + \mu + e$.
Phase $\varphi = b - \langle a, s \rangle = \mu + e$.

A bit is encoded as $\mu = \pm \frac{1}{8}$. In practice $\mathbb{T}$ is stored as $\mathbb{Z}_{2^{32}}$ or $\mathbb{Z}_{2^{64}}$.

Every gate ends with a bootstrapping: no levels, no depth limit.

A gate is a linear combination plus a sign

NAND

$$c = (0, \frac{1}{8}) - c_1 - c_2$$

has phase $\frac{3}{8}, \frac{1}{8}, \frac{1}{8}, -\frac{1}{8}$ for $(m_1, m_2) = (0, 0), (0, 1), (1, 0), (1, 1)$, plus the errors.

Bootstrapping returns a fresh encryption of $+\frac{1}{8}$ if the phase is in $(0, \frac{1}{2})$, of $-\frac{1}{8}$ otherwise.

Correct while the total error is below $\frac{1}{8}$. NAND is universal, so any circuit runs gate by gate.

- ① Round to $\mathbb{Z}_{2N}$: $\tilde{a}_i, \tilde{b}$, so $\tilde{\varphi} = \tilde{b} - \sum_i \tilde{a}_i s_i \approx 2N\varphi$.
- ② Test polynomial $v = \frac{1}{8}(1 + X + \dots + X^{N-1})$ in $\mathbb{T}[X]/(X^N + 1)$: the constant term of $X^{-k}v$ is $+\frac{1}{8}$ for $k < N$, $-\frac{1}{8}$ for $k \geq N$.
- ③ $ACC = X^{-\tilde{b}}v$; for each i , $ACC \leftarrow \text{CMux}(\text{BK}_i, X^{\tilde{a}_i}ACC, ACC)$, where BK_i is a GGSW encryption of s_i . Result: $X^{-\tilde{\varphi}}v$.
- ④ Extract the constant coefficient as an LWE ciphertext; key-switch back to s .

$\text{CMux}(C, d_1, d_0) = C \boxdot (d_1 - d_0) + d_0$ is GSW's product (Lecture 13): the accumulator sits in the decomposed slot, so noise grows additively.

Put the values of a function f in the coefficients of v .

One bootstrapping refreshes the noise *and* evaluates f on a message of a few bits.

Comparisons, ReLU, any lookup table: the non-polynomial functions that are expensive in BFV, BGV and CKKS.

Example — one NAND gate, bootstrapped

$N = 4$, so $2N = 8$.

- a. $m_1 = 1$ with error 0.01, $m_2 = 0$ with error -0.02 . Compute the phase of $(0, \frac{1}{8}) - c_1 - c_2$.
- b. Compute $\tilde{\varphi} = \lfloor 8\varphi \rfloor \bmod 8$. Which half of $\{0, \dots, 7\}$ is it in, and what bit comes out?
- c. Repeat for $m_1 = m_2 = 1$ with the same errors.

How TFHE differs

	BFV/BGV/CKKS	GSW	TFHE
Plaintext	many slots	a bit	a bit or few-bit integer
Ciphertext	ring pair	matrix	LWE vector
Multiply	tensor + relin	$G^{-1}(C_1)C_2$	gate = combination + bootstrap
Noise	levels	asymmetric	reset every gate
Bootstrap	rare, costly	costly	every gate, fast
Best for	arithmetic on many values	theory	comparisons, branching

Throughput (thousands of slots) against flexibility (any function, no depth to plan).

- 1 Bootstrapping GSW evaluates C_{Dec} in place of Π ; the output $\tilde{C}$ decrypts to the same bit with fresh noise.
- 2 C_{Dec} is one inner product $\sum_j C_{\ell j} s_j$ and a top-bit test. With a binary key: one AND layer plus an adder tree, depth $O(\log n + \log \log q)$.
- 3 ek holds an encryption of each key bit under that key; this needs circular security.
- 4 Sequential accumulation exploits GSW's asymmetric noise: $\|e_{\tilde{C}}\|_{\infty} \leq E(1 + g\ell)$, independent of the input's noise.
- 5 Bootstrapping works when $E(1 + g\ell) < q/4$: true with large margin at $q = 2^{60}$, false at $q = 2^{30}$.
- 6 TFHE bootstraps after every gate by blind rotation (GSW-type CMux steps); programmable bootstrapping also evaluates a lookup table.